



2.1 Kafka가 필요한가?

※ 상태	완료
≡ 다중 선택	Kafka

Yes! Kafka 필요하다! - Why? 준실시간 시스템 목표니까

cf) S3 (data lake)가 꼭 필요한 이유 : 백업

현재 방식(`logs.json` 파일에 계속 텍스트를 이어 붙이는 방식)을 그대로 유지할 경우 다음과 같은 문제가 발생합니다.

- **서버 용량 부족:** `로그 파일` 이 무한정 커지다가 결국 서버의 하드디스크 용량을 다 차지해 서버가 뿔어버립니다.
- **데이터 유실 위험:** 서버가 갑자기 꺼지거나 날아가면, 파일에 적혀있던 로그 데이터도 함께 날아갑니다.
- **확장성 문제:** 나중에 사용자가 많아져서 백엔드 서버를 2대, 3대로 늘리면, 각 서버마다 `logs.json` 파일이 따로 생겨서 하나로 합쳐서 보기 매우 힘들어집니다.

즉, "서버에 파일로 남긴다"는 현재 상태에 머무르면 안 되고, "이 파일을 주기적으로 안전한 곳(S3)으로 빼낸다"는 작업이 반드시 필요합니다.

Yes! Kafka 필요하다! - Why? 준실시간 시스템 목표니까

cf) S3 (data lake)가 꼭 필요한 이유 : 백업

1 정보 조사)

1. 백엔드에서 수집하는 데이터

- 1-1. 보통 백엔드에서는 운영 외의 데이터(이벤트 로그)를 어떻게 적재하나요?
- 1-2. 백엔드에서 수집하는 '비운영 데이터'의 종류 (이벤트 로그 외)
2. 백엔드에서 의도적으로 '안 다루는(버리는)' 데이터

2. 실시간 일때

2-1. 결정 - 실시간 (준실시간) 일때 ● → Kafka

Apache Kafka (실시간 대용량 스트리밍의 사실상 표준 / 엔터프라이즈급)

2-2. 실시간 (준실시간) 환경에서 Kafka의 대안들

대안 A - Redis Streams (가볍고 빠른 인메모리 방식 / 소규모 실시간 추천)

대안 B - AWS Kinesis Data Streams (클라우드 완전 관리형 / AWS 친화적 환경)

대안 C - RabbitMQ (정교한 라우팅이 필요한 전통적 메시지 브로커)

3. 실시간이 아닐때 ✖

1. 단순히 수집한 데이터를 S3에 적재하는 것이 목표라면, 현실적인 대안 3가지 (실시간X ⇒ Kafka X)

대안 A - 주기적 배치 업로드 (가장 쉬움, 현재 추천)

대안 B - Log Shipper (수집기) 사용 (실무 표준) 👍

대안 C - AWS Kinesis Data Firehose (클라우드 네이티브)

💡 요약 및 결론

2. '실시간'이 아니라면 Kafka는 아예 신경 안 써도 되나?
3. 그렇다면 실시간이 아닐 때의 수집 대안은 무엇이 있나?

2 4가지 대안 비교하기)

1 정보 조사)

1. 백엔드에서 수집하는 데이터

1-1. 보통 백엔드에서는 운영 외의 데이터(이벤트 로그)를 어떻게 적재하나요?

일단 이벤트 로그, 무조건 트래킹합니다. 사용자가 어디를 클릭했고, 장바구니에 뭘 담았는지 같은 '행동 데이터(Event Log)'는 기업의 핵심 자산이니까요.

하지만 절대 MySQL 같은 '운영 DB'에 넣지 않습니다. 사용자가 상품을 구경할 때마다 MySQL에 INSERT 를 때리면, 트래픽이 조금만 몰려도 DB가 뻘어버려서 진짜 중요한 결제(Transaction)를 못 하게 되는 대참사가 발생합니다.

그래서 현업에서는 보통 이렇게 분리합니다:

- 운영 데이터 (결제, 회원가입 등): MySQL, PostgreSQL 등 (안정성 최우선)
- 행동/로그 데이터 (클릭, 조회 등): 백엔드 서버가 1차적으로 파일(logs.json)로 남기거나 가벼운 수집기를 통해 S3 같은 '데이터 레이크(Data Lake)'로 무조건 빼냅니다

1-2. 백엔드에서 수집하는 '비운영 데이터'의 종류 (이벤트 로그 외)

현업에서는 유저의 클릭(이벤트 로그) 외에도 시스템과 비즈니스를 지탱하기 위해 다음과 같은 데이터들을 수집하여 S3나 외부 분석 시스템(Elasticsearch 등)으로 보냅니다.

- 시스템 성능 및 에러 로그 (Observability Data)
 - 내용: API 응답 시간, 서버 CPU/메모리 사용량, DB 쿼리 속도, 백엔드에서 발생한 에러(Exception) 스택 트레이스.
 - 이유: 유저가 결제 버튼을 눌렀는데 5초 이상 걸리거나 500 에러가 났을 때, 원인을 추적하기 위해 무조건 남깁니다. (보통 Datadog , Prometheus , ELK 스택을 씁니다.)
- 보안 및 감사 로그 (Audit Logs)
 - 내용: 누가 언제 어떤 IP로 로그인에 실패했는지, 관리자가 언제 어떤 유저의 권한을 변경했는지.
 - 이유: 해킹 시도를 탐지하거나, 내부 직원의 실수/부정을 추적하기 위한 법적 필수 데이터입니다.
- 외부 연동 데이터 (Third-party Webhooks)
 - 내용: 결제 대행사(PG사)에서 보내주는 결제 성공/실패 콜백, 카카오톡 알림톡 발송 성공 여부 등.

2. 백엔드에서 의도적으로 '안 다루는(버리는)' 데이터

모든 데이터를 다 수집하면 서버 비용이 폭발하기 때문에, 그리고 법적인 문제 때문에 현업에서도 절대 수집하지 않거나 프론트엔드 선에서 버리는 데이터들이 있습니다.

- 민감한 개인정보 및 금융 정보 (절대 수집/저장 불가)
 - 내용: 암호화되지 않은 낱것의 비밀번호(Plain-text password), 신용카드 전체 번호, CVC 번호.
 - 이유: 컴플라이언스(PCI-DSS, 개인정보보호법) 위반입니다. 비밀번호는 무조건 해싱해서 운영 DB에만 넣고 로그로는 절대 남기지 않습니다.
- 너무 세밀한 UI/UX 데이터 (노이즈 데이터)
 - 내용: 마우스 이동 궤적(X, Y 좌표), 스크롤을 몇 % 내렸는지, 화면을 터치한 압력 등.
 - 이유: 백엔드로 보내면 트래픽이 감당이 안 됩니다. 이런 데이터는 백엔드를 거치지 않고, 프론트엔드에서 뷰저블(Beusable), 핫자(Hotjar), 구글 애널리틱스(GA) 같은 외부 툴로 직접 싸버립니다. 백엔드 파이프라인에는 섞지 않습니다.

- 의미 없는 디버그(Debug) 로그

- 내용: 개발할 때나 보는 `console.log("여기까지 실행됨")`, `console.log("데이터: " + data)` 같은 자잘한 텍스트.
- 이유: 실제 운영(Production) 환경에서는 이런 로그를 끄거나 필터링해서 버립니다. 로그 저장소(S3, Elasticsearch) 비용만 잡아먹기 때문입니다.

2. 실시간 일때

2-1. 결정 - 실시간 (준실시간) 일때 → Kafka

준 실시간을 구현한다면, "Kafka는 오버스펙이다"라는 말은 취소!

쿠팡이나 넷플릭스처럼 사용자가 방금 본 상품/영화를 바탕으로 1~2초 만에 메인 화면 추천 리스트를 싹 바꿔버리는 아키텍처를 구현하려면 Kafka(또는 Kinesis)를 도입하는 것이 완벽한 정답입니다. (오히려 필수재에 가깝습니다!)

Apache Kafka (실시간 대용량 스트리밍의 사실상 표준 / 엔터프라이즈급)

쿠팡, 넷플릭스, 토스 등 대용량 실시간 처리가 필요한 모든 IT 기업이 사용하는 끝판왕 데이터 파이프라인

- 동작 방식: 백엔드 서버(Producer)가 이벤트 로그를 Kafka의 'Topic'이라는 공간에 쓰면, 이 데이터를 필요로 하는 여러 시스템(Consumer - 추천 엔진, 통계 서버, S3 백업 워커 등)이 각자의 처리 속도에 맞춰 독립적으로 실시간 데이터를 가져갑니다.
- 장점:
 1. 압도적인 처리량: 초당 수십만~수백만 건의 트래픽이 몰려도 서버가 뻘지 않고 든든하게 방어(Buffer)해 줍니다.
 2. 완벽한 확장성(Pub/Sub): 한 번 수집한 데이터를 A팀, B팀, C팀이 동시에 가져가도 시스템에 무리가 없습니다.
 3. 데이터 복구(Replay): Redis와 달리 데이터를 하드디스크에 안전하게 보관하므로, 추천 엔진 서버가 잠깐 죽었다가 살아나도 못 받은 데이터부터 다시 읽어올 수 있습니다.
- 단점: 설정, 운영, 유지보수 난이도가 극악에 가깝습니다.
 - 제대로 띄우려면 서버가 최소 3대(클러스터) 이상 필요하고
 - 메모리도 많이 먹기 때문에,
 - 소규모 프로젝트나 1인 개발에서는 인프라 관리하다가 지칠 수 있습니다.

2-2. 실시간 (준실시간) 환경에서 Kafka의 대안들

Kafka가 실시간 스트리밍의 '사실상 표준(De facto)'이긴 하지만, 프로젝트 규모나 팀의 인프라 환경에 따라 너무 무거울 수 있습니다.

이때 선택할 수 있는 대안에는 3가지 정도가 있습니다

대안 A - Redis Streams (가볍고 빠른 인메모리 방식 / 소규모 실시간 추천)

Kafka의 무거운 인프라가 부담스러울 때 현업에서 가장 많이 선택하는 '가성비 실시간 파이프라인'입니다.

- **동작 방식:** 백엔드 서버가 로그 데이터를 Redis 메모리에 밀어 넣습니다(**XADD**). 추천 엔진은 이 Stream을 실시간으로 구독(Subscribe)하고 있다가 0.1초 만에 데이터를 가져가서 추천 결과를 업데이트합니다.
- **장점:** Kafka에 비해 설치와 설정이 압도적으로 쉽고 가볍습니다. 데이터를 하드디스크가 아닌 메모리(RAM)에서 처리하므로 처리 속도가 극단적으로 빠릅니다.
- **단점:** 메모리 용량의 한계가 있기 때문에 데이터를 오래 보관할 수 없습니다. 따라서 실시간 처리를 끝낸 데이터는 주기적으로 지워주거나 **S3로 백업하는 별도의 로직이 필요합니다.**

대안 B - AWS Kinesis Data Streams (클라우드 완전 관리형 / AWS 친화적 환경)

클라우드 인프라에 익숙하고 서버 관리에 시간을 쏟고 싶지 않을 때 선택하는 완벽한 대체재입니다.

- **동작 방식:** 백엔드 서버가 Kafka 대신 AWS Kinesis API로 로그를 씁니다. Kinesis가 알아서 스트림을 만들고, 추천 엔진(Consumer)은 이 스트림에서 실시간으로 데이터를 읽어옵니다.
- **장점:** 직접 서버를 띄우고 관리할 필요가 없는 완전 관리형(Serverless)입니다.
 - Kinesis Firehose를 클릭 한 번으로 연결해 **실시간 처리는 Kinesis Streams가,**
 - **S3 백업은 Firehose가 동시에 알아서 처리**하게 만들 수 있습니다.
- **단점:** AWS 환경에 종속되며, 처리하는 데이터양과 커두는 시간만큼 지속적으로 클라우드 비용이 발생합니다.

대안 C - RabbitMQ (정교한 라우팅이 필요한 전통적 메시지 브로커)

대용량 로그 수집보다는, 마이크로서비스 간에 복잡한 규칙으로 실시간 메시지를 주고받아야 할 때 유리합니다.

- **동작 방식:** 백엔드가 이벤트를 중앙(Exchange)으로 보내면, 미리 설정한 규칙에 따라 여러 개의 큐(Queue)로 데이터가 똑똑하게 분배되고 추천 엔진이 이를 가져갑니다.
- **장점:** "A 상품 로그는 1번 큐로, B 상품 로그는 2번 큐로 보내라" 같은 정교한 라우팅 규칙을 짜는 데 있어서는 Kafka보다 훨씬 유연하고 강력합니다.
- **단점:** 트래픽이 초당 수만 건을 넘어가는 극단적인 대용량 처리에서는 Kafka보다 성능이 떨어집니다. 또한, 이미 가져간 데이터를 나중에 처음부터 다시 읽어보는 기능(Replay)이 제한적입니다.

3. 실시간이 아닐때 ❌

Kafka를 도입하는 것은 엄청난 오버스펙(Over-engineering)이 맞습니다

- 현업에서도 "로그를 S3에 저장한다"는 목적 하나만으로 Kafka를 구축하지는 않습니다.
- 시뮬레이터를 통해 데이터 양을 늘린다고 하더라도, 초당 수만~수십만 건의 실시간 처리가 필요한 게 아니라면 굳이 유지보수가 까다로운 Kafka를 선택할 이유가 없습니다.

1. 단순히 수집한 데이터를 S3에 적재하는 것이 목표라면, 현실적인 대안 3가지 (실시간X ⇒ Kafka X)

단순히 로그를 S3에 적재하는 것이 목표라면,

cf) "서버에 파일로 남긴다"는 현재 상태에 머무르면 안 되고, "이 파일을 주기적으로 안전한 곳(S3)으로 빼낸다"는 작업이 반드시 필요

대안 A - 주기적 배치 업로드 (가장 쉬움, 현재 추천)

가장 단순무식하지만 초기에 가장 확실한 방법입니다.

- **동작 방식:** 백엔드 서버에 `node-cron` 같은 스케줄러를 달아서, "1시간마다(또는 로그 파일이 10MB가 될 때마다) 현재 `logs.json` 파일을 AWS S3로 전송하고, 기존 파일은 비운다"는 로직을 직접 코딩합니다.
- **장점:** 추가적인 인프라나 프로그램 설치 없이 Node.js 코드 몇 줄로 끝납니다.
- **단점:** 서버가 중간에 죽으면 S3로 미처 보내지 못한 최근 1시간 치 로그가 날아갈 수 있습니다.

대안 B - Log Shipper (수집기) 사용 (실무 표준) 👍

현업에서 가장 많이 쓰는 '가성비' 인프라 구성입니다. **Fluent Bit, Logstash, Vector** 같은 가벼운 로그 수집기 프로그램을 서버에 띄웁니다.

- **동작 방식:** 이 프로그램이 `logs.json` 파일에 새로운 줄이 추가되는지 실시간으로 감시(tail)하다가, 알아서 버퍼링을 거쳐 S3로 안전하게 씁니다.
- **장점:** 서버 코드(Node.js)를 수정할 필요가 없고, 전송 실패 시 재시도 기능 등 안정성이 매우 뛰어납니다. Kafka 보다 설정이 훨씬 쉽고 가볍습니다.

대안 C - AWS Kinesis Data Firehose (클라우드 네이티브)

AWS 생태계를 적극적으로 쓴다면 Kafka의 완벽한 대체재입니다.

- **동작 방식:** 백엔드 서버에서 파일에 로그를 쓰는 대신, AWS의 Kinesis Firehose API로 데이터를 직접 씁니다. Firehose가 알아서 1분~5분 단위로 데이터를 뭉쳐서 S3에 예쁘게 저장해 줍니다.
- **장점:** 내가 관리할 서버 인프라(Kafka 등)가 아예 없습니다(Serverless). 클릭 몇 번으로 세팅이 끝납니다.
- **단점:** AWS 사용 요금이 조금 발생합니다.

💡 요약 및 결론

Kafka는 여러 개의 마이크로서비스가 얹혀 있고, 하나의 로그를 A팀, B팀, C팀이 동시에 실시간으로 가져가서 각자 다른 처리를 해야 할 때(Pub/Sub 구조) 진가를 발휘합니다. **단순 로그 저장용으로는 절대 쓰지 마세요.**

지금 단계에서는 무거운 인프라 구성에 시간을 쏟기보다는, 대안 A(Node.js 스케줄러로 직접 S3 업로드)나 대안 B(Fluent Bit 도입)로 가볍게 파이프라인을 뚫어두고, 핵심 비즈니스 로직과 데이터 시뮬레이터를 만드는 데 집중하시는 것을 강력히 권장합니다.

2. '실시간'이 아니라면 Kafka는 아예 신경 안 써도 되나?

결론부터 말씀드리면, '실시간' 조건이 빠진다면 Kafka를 도입할 매력도가 뚝 떨어지는 것이 사실입니다.

하지만 실시간이 아닌데도 대기업들이 Kafka를 쓰는 경우가 있습니다. 그 이유는 크게 두 가지입니다.

- 첫째, 엄청난 트래픽의 '충격 흡수 버퍼(Buffer)' 역할:

이벤트나 기획전으로 갑자기 10만 명이 몰려서 클릭 로그가 초당 수만 개씩 쏟아진다고 가정해 볼게요.

→ DB나 다른 서버가 이걸 다 받아내면 죽어버립니다.

이때 Kafka가 중간에서 "일단 내 배 속에 다 던져!" 하고 엄청난 속도로 로그를 집어삼킨 뒤, 뒷단에 있는 시스템들이 본인들 소화 속도에 맞춰 천천히 가져가게 해 줍니다.

- 둘째, 부서 간 데이터 파이프라인의 '중앙 허브':

A팀은 추천 알고리즘 훈련용으로 그 로그가 필요하고, B팀은 마케팅 통계용으로 필요하고, C팀은 S3 백업용으로 필요합니다.

Kafka에 로그를 싣두면, 각 팀이 눈치 보지 않고 원하는 시간에 마음대로 데이터를 퍼갈 수 있습니다 (Pub/Sub 구조).

요약하자면:

트래픽이 초당 수천~수만 단위로 미친 듯이 몰리거나, 데이터를 가져다 쓰는 팀이 여러 개가 아니라면 비실시간 환경에서 Kafka는 확실히 오버스펙입니다.

3. 그렇다면 실시간이 아닐 때의 수집 대안은 무엇이 있나?

실시간 처리(초 단위 반응)가 필요 없고, 1시간이나 하루 단위로 데이터를 모아서 처리(Batch)해도 된다면, 파이프라인은 훨씬 가볍고 우아해집니다.

- 가장 완벽한 표준 (Log Shipper + S3): 이전 답변에서 말씀드린 Fluent Bit이나 Logstash가 현업에서 가장 사랑받는 정답

서버에 아주 가벼운 (수집기) 프로그램 하나 띄워두면, logs.json 에 데이터가 쌓일 때마다 알아서 압축하고 묶어서 1분 ~5분 주기로 S3에 던져줍니다. (Kafka 설치에 비하면 난이도가 1/10 수준입니다.)

- AWS 생태계를 쓴다면 (AWS SQS): 만약 약간의 버퍼(충격 흡수) 역할이 필요한 한데 Kafka는 너무 무겁다면, AWS의 SQS(Simple Queue Service)를 씁니다.

설정할 것도 없고, 가격도 매우 싸고, 웬만한 트래픽은 다 막아주는 아주 훌륭한 큐 시스템입니다.

2 4가지 대안 비교하기)

항목 \ 대안	Kafka	Redis Streams	AWS Kinesis	RabbitMQ
S3 백업 + 추천엔진 동시 전송 (Pub/Sub)	최상 (각자 완벽히 독립적으로 가져감)	보통 (가능은 하나 S3용 코드를 직접 짜야 함)	최상 (Firehose가 S3 백업을 알아서 해줌)	우수 (라우팅은 좋으나 데이터 보관에 취약)
추천 모델 서버가 죽었을 때 (데이터 복구)	가능 (못 읽은 과거 데이터부터 다시 읽기 가능)	제한적 (메모리 꽉 차서 지워지면 복구 불가)	가능 (과거 데이터 Replay 지원)	어려움 (가져간 데이터는 보통 바로 사라짐)
설정 및 관리 난이도	극상 (가장 어려움)	하 (아주 쉬움)	중 (AWS 콘솔 세팅 필요)	중 (세팅 자체는 무난함)

항목 \ 대안	Kafka	Redis Streams	AWS Kinesis	RabbitMQ
비용 (클라우드 이전 시)	높음 (서버 여러 대 필요)	중간 (메모리 인스턴스 비용)	사용한 만큼만 청구 (유연함)	중간
포트폴리오 매력도 (이력서)	★★★★★ (압도적 1티어)	★★★	★★★★	★★